read_files() vs COPY INTO vs Auto Loader

Databricks Data Engineer Associate · Course 1 “Data Ingestion with Lakeflow Connect”, slides 29–40 · notebooks 02A, 02B, A2

Big idea first

- **read_files()** is a **function** that reads files into a table shape. It does **not** remember anything by itself. With `CREATE TABLE … AS SELECT` it is **batch** (rereads all files). With `FROM STREAM read_files(…)` it becomes **Auto Loader**.
- **COPY INTO** is a **SQL command** for **incremental batch**: it remembers which files it loaded and skips them (**idempotent**). The course calls it *legacy*.
- **Auto Loader** (`cloudFiles`) is **incremental batch or streaming**, built on **Structured Streaming**. A **checkpoint** remembers processed files. Scales to millions of files.

Side-by-side

	CTAS + read_files() / spark.read	COPY INTO	Auto Loader
Ingestion type	Batch: reads **all** files every run	Incremental batch: only **new** files	Incremental batch **or** streaming
Best for	Small data, one-time / ad-hoc loads	Thousands of files, scheduled loads	Millions of files per hour, near real-time, billions for backfill
Interface	SQL (CTAS) · Python `spark.read`	SQL only	Python `spark.readStream.format("cloudFiles")` · SQL streaming tables / pipelines (`STREAM read_files`)
Remembers loaded files?	No	Yes, in the target table’s metadata	Yes, in the **checkpoint**
Idempotent (run twice = no duplicates)?	**No**	**Yes** · `'force'='true'` reloads anyway	**Yes** (exactly-once per file)
Schema	Inferred or given with `schema =>`	Evolve with `COPY_OPTIONS ('mergeSchema'='true')`	Automatic inference + evolution; stored at `cloudFiles.schemaLocation`
Bad / mismatched data	`_rescued_data` column (on by default)	Error on schema mismatch unless mergeSchema	`_rescued_data` column
Latency	High	Moderate (scheduled)	Low or high, depending on trigger
Ease of use	Simple	Simple, SQL-based	Intermediate → advanced

Syntax to recognize

CTAS + read_files()

Batch. Delta table by default.

```
CREATE TABLE users_bronze AS
SELECT *,
  _metadata.file_name AS source_file
FROM read_files(
  '/Volumes/cat/sch/raw/users',
  format => 'csv',
  header => true, sep => '|',
  schema => 'id INT, email STRING',
  rescueddatacolumn => '_rescued_data');
```

Quick look: `SELECT * FROM csv.`/Volumes/…``

COPY INTO

Target can be empty and have no schema.

```
CREATE TABLE IF NOT EXISTS users_bronze;

COPY INTO users_bronze
FROM '/Volumes/cat/sch/raw/users'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true',
                'sep' = '|')
COPY_OPTIONS ('mergeSchema' = 'true');
```

FORMAT_OPTIONS = how to *parse*. **COPY_OPTIONS** = how COPY INTO *behaves* (mergeSchema, force).

Auto Loader

Python (`cloudFiles` = Auto Loader):

```
(spark.readStream
   .format("cloudFiles")
   .option("cloudFiles.format", "json")
   .option("cloudFiles.schemaLocation", chk)
   .load("/Volumes/cat/sch/raw/events")
 .writeStream
   .option("checkpointLocation", chk)
   .trigger(availableNow=True)
   .toTable("events_bronze"))
```

SQL (streaming table — `STREAM` = Auto Loader):

```
CREATE OR REFRESH STREAMING TABLE events_bronze
SCHEDULE EVERY 1 HOUR      -- optional
AS SELECT *
FROM STREAM read_files(
  '/Volumes/cat/sch/raw/events',
  format => 'json');

REFRESH STREAMING TABLE events_bronze;
```

Which one? (exam keywords)

“one-time”, “ad-hoc”, “small dataset”, “explore files”	**CTAS + read_files**
“SQL only”, “thousands of files”, “idempotent”, “skip loaded files”, “retriable”	**COPY INTO**
“millions of files”, “near real-time”, “streaming”, “schema drift / new columns appear”	**Auto Loader**
“streaming table in Databricks SQL / pipeline”	**STREAM read_files** (= Auto Loader)

Exam traps

- ⚠ Running CTAS + read_files again rereads **everything**. It is not incremental.
- ⚠ COPY INTO run twice adds **0** rows the second time, unless `'force'='true'`.
- ⚠ New columns in COPY INTO → `mergeSchema` goes in **COPY_OPTIONS**.
- ⚠ `format("cloudFiles")` = Auto Loader. `cloudFiles.format` = the file type.
- ⚠ In SQL, the **STREAM** keyword is what turns read_files into Auto Loader.
- ⚠ Auto Loader needs a **checkpoint** (and a schemaLocation for inference).
- ⚠ `trigger(availableNow=True)` = process all new files, then stop (incremental batch). `processingTime` = keep running.

Ingestion methods (slides 18–20): **Batch** = CTAS, spark.read · **Incremental batch** = COPY INTO, Auto Loader with a timed trigger, CREATE OR REFRESH STREAMING TABLE · **Streaming** = Auto Loader with a continuous trigger, pipelines in continuous mode.
Practise: `notebooks/lab1_ingestion.ipynb` exercises 1b (CTAS), 1c (COPY INTO), 1d (Auto Loader idea).